

LC 1140: Stone Game II

Target Companies: Uber, Amazon

Game Rules & Constraints

- Aadya starts first.
- In one turn, a player can take all stones in the **first X remaining piles**.
- The condition for X is: $1 \leq X \leq 2M$, where M starts from 1.
- After a turn, M updates dynamically: $M = \max(M, X)$
- Both Aadya and Bob play optimally. We need the **max stones Aadya can get**.

Dynamic Programming State

So this time we define our DP state as:

$dp[i][M][player]$

- $dp[i][M][0]$ → curr player
- $dp[i][M][1]$ → other player
- i → start idx
- M → dynamic and will change every turn

State Transitions & Scoring

At (i, M) , the curr player chooses $(i, i+1, i+2, \dots, i+X-1)$.

Hence Score is: $Sum(i, i+X-1)$.

Formula Updates:

- $\text{newM} = \max(M, X)$
- $\text{curr player score} = \text{the took sum} + \text{dp}[i+X][\text{newM}][1]$
- $\text{other player score} = \text{dp}[i+X][\text{newM}][0]$

The valid range for choices is: $X \in (1, \min(2*M, n-i))$.

Finally, the answer will be: $\text{ans} = \text{dp}[0][1][0]$.

Source Code

```

class Solution {
public:
    int stoneGameII(vector<int>& piles) {

        int n=piles.size();
        vector<int> pref(n+1,0);

        //for range sums

        for(int i=0;i<n;i++){
            pref[i+1]=pref[i]+piles[i];
        }

        int maxm=(n+1)/2;
        //because x==2M at max , and x can be at max n

        //dp[i][m][0] or dp[i][m][1]
        //0 if currplayer
        //1 if otherplayer
        vector<vector<vector<int>>> dp(n+1,vector<vector<int>>
(maxm+1,vector<int>(2,0)));
        //agar currplayer x tak uthayega to next will be [i+x][newM]
[1]

        //x tak uthne ke liye prefix sum use kr lenge
        //x depends on i+x , so top down dp

        for(int i=n-1;i>=0;i--){
            for(int m=maxm;m>=1;m--){

                int best=0;
                int otherbest=0;

                for(int x=1;x<=min(2*m,n-i);x++){ //n-i to respect
limit , cause we are accesing i+x

                                                                    //i+x<n+1
                                                                    //so i+2*m<n+1
                                                                    //i+2*m<=n
                                                                    //2*m<=n-i

                int took=pref[i+x]-pref[i]; ///aise to i+x-1 hota
lekin 1 indexxed he
                int newm=max(m,x);

                newm=min(newm,maxm); //maxm se bada nhi hona

```

chahiye

```
        int currscore=took+dp[i+x][newm][1];
        int otherscore=dp[i+x][newm][0];

        if(currscore>best){
            best=currscore;
            otherbest=otherscore;
        }
    }

    dp[i][m][0]=best;
    dp[i][m][1]=otherbest;
}

return dp[0][1][0];
}

};
```